bond_pricing

October 30, 2026

1 Corporate bond pricing in OSEM

This notebook walks through how OSEM treats a corporate bond position: from raw input data, to projected cash flows, to a calibrated market-consistent price. Each corporate bond is defined by its characteristic, the market conditions such as spread and price.

On a very high level:

1) Each corporate bond is saved into a CorpBond object.
2) A portfolio of CorpBond objects is then saved into the CorpBondPortfolio.
3) Based on the CorpBondPortfolio and market conditions, the cash flows resulting from each bond are reconstructed into a matrix
4) Based on the cash flows and current market conditions, the z-spread is calculated using the bisection algorithm to return the current market price

1.1 Table of contents

1. Methodology
   - Notation
   - Step 1 — Cash flow dates · Step 2 — Date fractions · Step 3 — Cash flow amounts
   - Step 4 — Pricing off the risk-free curve · Step 5 — Calibrating the z-spread
   - Step 6 — Calibrating the whole portfolio · Step 7 — Use in the full OSEM run
   - Generating new bonds
2. Setup
   - Environment and imports
   - Run parameters
3. The bond portfolio
   - Loading the bonds
   - Which fields drive the price
   - Sector spread input
4. Steps 1 and 3 in numbers: cash flow dates and amounts
   - Coupon and notional flows
   - Cash flow matrices
5. Step 4 in numbers: pricing off the risk-free curve
   - The risk-free curve
   - Pricing state at the modelling date
   - Risk-free price: the gap the z-spread has to close
   - Price at the input z-spread
6. Step 5 in numbers: calibrating one bond

7. Step 6 in numbers: calibrating the whole portfolio
8. Appendix: input data
    - Parameters.csv · EIOPA RFR files
    - Bond_Portfolio.csv · Sector_Spread.csv

1.2 1. Methodology

How OSEM turns a bond description into a price, step by step. No code in this section; the sections that follow run each step on the input portfolio.

1.2.1 Notation

This table defines the mathematical notation that will be used through the rest of the workbook

Symbol	Meaning
MD	modelling date
VD	valuation date, the date cash flows are discounted to ($VD = MD$ at $t = 0$)
E	end of the modelling window (`settings.end_date`)
I	issue date
M	maturity date
L	last cash flow date, $\min(E, M)$
N	notional amount
c	coupon rate, paid on **each** coupon date
f	coupon frequency (payments/year)
MV	market value (price) at the modelling date
z	bond-specific z-spread
$y(t)$	risk-free spot yield for maturity t, from the calibrated EIOPA curve

A note on f. The frequency sets only the *spacing* of the coupon dates, through `relativedelta(months=12 // f)`. It does **not** scale the coupon amount: c is the rate paid on each coupon date, not an annualised rate, so a bond with $c = 0.02$ and quarterly frequency pays 2% of the notional four times a year. A bond quoted with an annual coupon c_{ann} paid f times a year is entered in `Bond_Portfolio.csv` with `Coupon_Rate` $= c_{ann}/f$.

1.2.2 Step 1 — Cash flow dates

Every bond produces two kinds of cash flow: coupons and the return of notional.

A bond still alive when the modelling window closes is redeemed there at par, so both kinds of cash flow stop at

$$L = \min(E, M)$$

Notional repayment date:

$$t_M^d = L$$

Coupon dates:

$$I < t_1^d < t_2^d < \cdots < t_k^d \leq L$$

The coupon grid is anchored on the issue date and stepped by $12/f$ months. Payments falling before MD are dropped, since only future cash flows are priced.

In numbers: section 4.

1.2.3 Step 2 — Date fractions relative to the valuation date

Only cash flows after the valuation date matter. Each remaining cash flow date is converted to a year fraction from it:

$$t_i = \frac{t_i^d - VD}{365.25}$$

At the modelling date $VD = MD$, which is the case this notebook demonstrates. In a full run the valuation date moves forward with every period and the fractions are re-measured from the new date.

This is what lets the discounting step treat every cash flow generically as "an amount, this many years from now."

Step 2 runs inside `price_bond` *and has no table of its own: the year fractions are the maturities at which the discount factors in section 5 are evaluated.*

1.2.4 Step 3 — Cash flow amounts

Notional cash flow (paid once, at maturity):

$$cf_M = N$$

Coupon cash flows (paid at each coupon date):

$$cf_i = N \cdot c$$

In numbers: section 4.

1.2.5 Step 4 — Pricing off the risk-free curve

Before any bond-specific adjustment, each cash flow can be discounted purely with the risk-free spot curve calibrated in `CurvesClass`:

$$MV_{\text{rf}} = \sum_{i=1}^{k} \frac{cf_i}{(1 + y(t_i))^{t_i}} + \frac{cf_M}{(1 + y(t_M))^{t_M}}$$

This is *not* the bond's actual market price — it ignores credit/liquidity risk entirely. The gap between it and the real `Market_Price` from the input file is exactly what the z-spread in Step 5 is calibrated to close.

In numbers: section 5.

1.2.6 Step 5 — Calibrating the z-spread to match the market price

OSEM finds a single constant z added to the risk-free yield at every maturity such that the discounted cash flows reproduce the bond's observed market price exactly:

$$MV = \sum_{i=1}^{k} \frac{cf_i}{(1 + y(t_i) + z)^{t_i}} + \frac{cf_M}{(1 + y(t_M) + z)^{t_M}}$$

There is no closed-form solution for z, so OSEM finds it numerically with a **bisection search** (`bisection_spread`) over a bracket $[z_{\text{start}}, z_{\text{end}}]$, which `calibrate_bond_portfolio` fixes at $[-0.2, 0.2]$.

The bracket has to contain a solution. The price falls monotonically in z, so a root exists only if MV lies between the prices at the two ends of the bracket. `bisection_spread` checks this before iterating and raises a `ValueError` naming the bond when it fails. Without that check the search never sees a sign change, walks z_{start} all the way up to z_{end}, and returns the bracket bound as though it were a calibrated spread. A market price outside the bracket usually means either that the bracket is too narrow for a genuinely distressed bond, or that `Coupon_Rate` was entered as an annualised rate (see the note on f above).

What `precision` measures. Once bracketed, the search stops when the bracket is narrower than `precision`, that is $(z_{\text{end}} - z_{\text{start}})/2 <$ `precision`. This is a tolerance on the *spread*, not on the price. `calibrate_bond_portfolio` hardcodes `precision` $= 10^{-8}$, which leaves a residual of order 10^{-6} in price on the bonds in this portfolio. The two bracket endpoints are the exception: each is returned straight away if it already prices within `precision` of MV.

In numbers: section 6.

1.2.7 Step 6 — Calibrating the whole portfolio

`calibrate_bond_portfolio` simply repeats Step 5's bisection for every bond in the portfolio, storing one z per asset in `zspread_df`.

In numbers: section 7.

1.2.8 Step 7 — How this is used in the full OSEM run

This notebook prices bonds once, at $t = 0$. In a full run (`main.py`):

1. The portfolio's z-spreads are calibrated **once**, at the modelling date (`calibrate_bond_portfolio`, called in `main.py` before the main loop starts).
2. In every subsequent period of the simulation, `price_bond_portfolio` is called again with that *same, fixed* z-spread but the *period's* point on the projected yield curve (the bond repricing step of the main loop) — so the bond's price moves only because the risk-free curve and remaining cash flows change, not because its credit spread is re-estimated.

This notebook only demonstrates the fixed-income leg in isolation; the full run also handles equities, cash, liabilities/unit-linked policies, and portfolio rebalancing each period.

1.2.9 Generating new bonds (not yet implemented)

OSEM's documentation also describes a process for generating *new* corporate bonds at each future period (to replace maturing debt), calibrating their coupon so they price at par:

$$1 = \sum_{i=1}^{k} \frac{dy}{(1 + y(t_i) + c + s + ss)^{t_i}} + \frac{1}{(1 + y(t_M) + c + s + ss)^{t_M}}$$

This is not yet implemented in `BondClasses.py` / this notebook — see `Documentation/OSEM_Documentation_draft.ipynb` (cells 84–99) for the full methodology.

1.3 2. Setup

Prepares the environment and loads the run parameters. Nothing in this section is specific to bonds.

1.3.1 Environment and imports

Locate the repository root (the folder holding `ALM.ini`), make the `osem` package importable from `src/`, and import the parts of OSEM this notebook uses.

[1]:
```
import os
import sys
import numpy as np
import pandas as pd
import datetime as dt
from pathlib import Path

# Walk up to the repository root, the folder holding ALM.ini
base_folder = next(path for path in [Path.cwd(), *Path.cwd().parents] if (path /
 ↪ "ALM.ini").is_file())
sys.path.insert(0, str(base_folder / "src"))
os.chdir(base_folder)
base_folder = os.getcwd()  # Get current working directory

from osem.CurvesClass import Curves
from osem.ImportData import import_SWEiopa, get_corporate_bonds,␣
 ↪get_configuration, get_settings
from osem.BondClasses import *
from osem.ConfigurationClass import Configuration
from osem.MainLoop import create_cashflow_dataframe
```

1.3.2 Run parameters

`ALM.ini` gives the location of every input file:

[2]:
```
conf: Configuration
conf = get_configuration(os.path.join(base_folder, "ALM.ini"), os)
```

[3]:
```
parameters_file = conf.input_parameters
cash_portfolio_file = conf.input_cash_portfolio
bond_portfolio_file = conf.input_bond_portfolio
```

[4]:
```
paramfile = pd.read_csv("Input/Parameters.csv").set_index("Parameter")
```

The parameter file is:

---

[5]:
```
display(paramfile)
```

Parameter	Value
EIOPA_param_file	Input/Param_no_VA.csv
EIOPA_curves_file	Input/Curves_no_VA.csv
country	Slovenia
run_type	Risk Neutral
n_proj_years	50
Precision	1E-10
Tau	0.0001
compounding	-1
Modelling_Date	29/04/2023
liability_mode	cashflow
random_seed	42

The settings object holds data about file locations, information about the run settings and model parameters such as modelling date.

[6]:
```
settings = get_settings(parameters_file)
```

The run that the rest of the notebook prices against:

---

[7]:
```
run_settings = pd.DataFrame(
    # Rendered as strings: in a mixed-type column pandas applies one float␣
 ↪format to every
    # numeric value, which would show a precision of 1e-10 as 0.0
    {"Value": [str(settings.modelling_date),
               str(settings.end_date),
               str(settings.n_proj_years),
               settings.country,
               f"{settings.precision:g}",
               f"{settings.tau:g}"]},
    index=["Modelling date",
           "End of modelling window",
           "Projection years",
           "Country",
```

```
            "Precision (curve calibration)",
            "Tau"],
)

display(run_settings)
```

	Value
Modelling date	2023-04-29
End of modelling window	2073-04-29
Projection years	50
Country	Slovenia
Precision (curve calibration)	1e-10
Tau	0.0001

1.4 3. The bond portfolio

1.4.1 Loading the bonds

A Python generator reads the bond portfolio file and encodes it into a dictionary based on the asset id. Each asset id contains a CorpBond object describing a single fixed income position.

[8]:
```
bond_input_generator = get_corporate_bonds(bond_portfolio_file)
bond_input = {corp_bond.asset_id: corp_bond for corp_bond in␣
 ↪bond_input_generator}
```

As an example, a single corporate bond in a CorpBond would look something like:

---

[9]:
```
display(bond_input[1234])
```

```
CorpBond(asset_id=1234, nace='A1.4.5', issuer=None, issue_date=datetime.
 ↪date(2021, 12, 3), maturity_date=datetime.date(2026, 12, 12), coupon_rate=0.
 ↪03, notional_amount=100.0, spread_country=0.0, spread_sector=0.0, zspread=0.
 ↪01, spread_stress=0.0, frequency=1, recovery_rate=0.4, default_probability=0.
 ↪03, units=100.0, market_price=94.0)
```

`CorpBondPortfolio` wraps a dictionary of these objects, keyed by asset id, and provides the portfolio-level methods used in the rest of the notebook.

[10]:
```
bond_portfolio = CorpBondPortfolio(bond_input)
```

The whole fixed income portfolio at a glance. Note that `Annual coupon` is derived, not an input: it is `Coupon rate` times `Frequency`, because the coupon rate is the rate paid on each coupon date rather than an annualised one.

---

[11]:
```
portfolio_overview = pd.DataFrame(
    [{"Maturity": bond.maturity_date,
      "Notional": bond.notional_amount,
      "Coupon rate": bond.coupon_rate,
      "Frequency": int(bond.frequency),
      "Annual coupon": bond.coupon_rate * int(bond.frequency),
      "Units": bond.units,
      "Market price": bond.market_price}
     for bond in bond_portfolio.corporate_bonds.values()],
    index=list(bond_portfolio.corporate_bonds),
)

display(portfolio_overview)
```

	Maturity	Notional	Coupon rate	Frequency	Annual coupon	Units	\
1234	2026-12-12	100.0	0.030000	1	0.03	100.0	
2889	2028-12-12	100.0	0.025000	2	0.05	120.0	
31	2025-12-03	100.0	0.003333	12	0.04	100.0	
1	2030-06-30	100.0	0.005000	2	0.01	120.0	
2	2035-12-31	100.0	0.010000	1	0.01	100.0	
3	2026-12-12	100.0	0.030000	1	0.03	120.0	
4	2028-12-12	100.0	0.025000	2	0.05	100.0	
5	2025-12-03	100.0	0.003333	12	0.04	120.0	
6	2026-12-12	100.0	0.030000	1	0.03	100.0	
7	2028-12-12	100.0	0.025000	2	0.05	120.0	
8	2025-12-03	100.0	0.003333	12	0.04	100.0	
9	2026-12-12	100.0	0.030000	1	0.03	120.0	
10	2028-12-12	100.0	0.025000	2	0.05	100.0	
11	2025-12-03	100.0	0.003333	12	0.04	120.0	
12	2026-12-12	100.0	0.030000	1	0.03	100.0	
13	2028-12-12	100.0	0.025000	2	0.05	120.0	
14	2025-12-03	100.0	0.003333	12	0.04	100.0	
15	2026-12-12	100.0	0.030000	1	0.03	120.0	
16	2028-12-12	100.0	0.025000	2	0.05	100.0	
17	2025-12-03	100.0	0.003333	12	0.04	120.0	
18	2026-12-12	100.0	0.030000	1	0.03	100.0	
19	2028-12-12	100.0	0.025000	2	0.05	120.0	
20	2025-12-03	100.0	0.003333	12	0.04	100.0	
21	2026-12-12	100.0	0.030000	1	0.03	120.0	
22	2028-12-12	100.0	0.025000	2	0.05	100.0	
23	2025-12-03	100.0	0.003333	12	0.04	120.0	
24	2026-12-12	100.0	0.030000	1	0.03	100.0	
25	2028-12-12	100.0	0.025000	2	0.05	120.0	
26	2025-12-03	100.0	0.003333	12	0.04	100.0	
27	2026-12-12	100.0	0.030000	1	0.03	120.0	
28	2028-12-12	100.0	0.025000	2	0.05	100.0	
29	2025-12-03	100.0	0.003333	12	0.04	120.0	
30	2030-06-30	100.0	0.005000	2	0.01	10.0	

	Market price
1234	94.0
2889	92.0
31	96.0
1	90.0
2	85.0
3	94.0
4	92.0
5	96.0
6	94.0
7	92.0
8	96.0
9	94.0
10	92.0
11	96.0
12	94.0
13	92.0
14	96.0
15	94.0
16	92.0
17	96.0
18	94.0
19	92.0
20	96.0
21	94.0
22	92.0
23	96.0
24	94.0
25	92.0
26	96.0
27	94.0
28	92.0
29	96.0
30	90.0

1.4.2 Which fields drive the price

Only seven of the `CorpBond` fields enter the price. The rest are loaded from the portfolio file but play no part in it; credit risk is carried by the z-spread alone.

`CorpBond` field	CSV column	Used in pricing	Role
`asset_id`	`Asset_ID`	–	Key of the portfolio dictionary and row index of every DataFrame

CorpBond field	CSV column	Used in pricing	Role
issue_date	Issue_Date	Yes	Anchors the coupon grid (I, Step 1)
maturity_date	Maturity_Date	Yes	Notional repayment date (M, Step 1)
frequency	Frequency	Yes	Spacing of the coupon dates, $12/f$ months (f, Step 1)
notional_amount	Notional_Amount	Yes	Notional repaid at L (N, Step 3)
coupon_rate	Coupon_Rate	Yes	Coupon paid on each coupon date, as a fraction of N (c, Step 3)
market_price	Market_Price	Yes	Calibration target (MV, Step 5)
zspread	Z_Spread	Starting value only	Seeds zspread_df; overwritten by the calibrated z-spread (Step 6)
units	Units	No	Number of bonds held; scales the cash flows in the main loop, not the per-unit price
nace	NACE	No	Sector code; would link the bond to Sector_Spread.csv, not used in the POC
spread_country, spread_sector, spread_stress	Spread_Country, Spread_Sector, Spread_Stress	No	Loaded but not used in bond pricing
recovery_rate, default_probability	Recovery_Rate, Default_Probability	No	Loaded but not used; there is no default model yet
issuer	–	No	Not read from the CSV; always None

1.4.3 Sector spread input

The sector spread file lists the NACE classification and the spread components that go with it. In this proof of concept the spread is taken directly from the bond input file, so spreadfile is loaded here for reference only.

[12]:
```
spreadfile = pd.read_csv("Input/Sector_Spread.csv")
spreadfile.index = spreadfile["NACE"]
del spreadfile["NACE"]
```

1.5 4. Steps 1 and 3 in numbers: cash flow dates and amounts

This section builds the coupon and notional schedules of Step 1 and the amounts of Step 3 for every bond. Step 2, the conversion to year fractions, happens inside `price_bond` when the flows are discounted in the next section.

1.5.1 Coupon and notional flows

The coupon payments and the return of the notional are simulated separately.

`create_coupon_flows` returns the coupon dates and amounts of every bond, as a dictionary keyed by asset id:

[13]:
```
coupon_flows = bond_portfolio.create_coupon_flows(settings.modelling_date,␣
↪settings.end_date)
```

`create_maturity_flows` returns the notional repayments in the same shape:

[14]:
```
notional_flows = bond_portfolio.create_maturity_flows(terminal_date=settings.
↪end_date)
```

A compact view of the same two dictionaries, one row per bond. The last coupon never falls after the date the notional is repaid, which is $L = \min(E, M)$ from Step 1:

---

[15]:
```
coupon_schedule = pd.DataFrame({
    "Coupons": {aid: len(flows) for aid, flows in coupon_flows.items()},
    "Coupon amount": {aid: next(iter(flows.values())) for aid, flows in␣
↪coupon_flows.items()},
    "First coupon": {aid: min(flows) for aid, flows in coupon_flows.items()},
    "Last coupon": {aid: max(flows) for aid, flows in coupon_flows.items()},
    "Notional repaid": {aid: min(flows) for aid, flows in notional_flows.
↪items()},
})

display(coupon_schedule)
```

	Coupons	Coupon amount	First coupon	Last coupon	Notional repaid
1234	4	3.000000	2023-12-03	2026-12-03	2026-12-12
2889	12	2.500000	2023-06-03	2028-12-03	2028-12-12
31	32	0.333333	2023-05-03	2025-12-03	2025-12-03
1	15	0.500000	2023-06-30	2030-06-30	2030-06-30
2	13	1.000000	2023-12-31	2035-12-31	2035-12-31
3	4	3.000000	2023-12-03	2026-12-03	2026-12-12
4	12	2.500000	2023-06-03	2028-12-03	2028-12-12
5	32	0.333333	2023-05-03	2025-12-03	2025-12-03
6	4	3.000000	2023-12-03	2026-12-03	2026-12-12
7	12	2.500000	2023-06-03	2028-12-03	2028-12-12
8	32	0.333333	2023-05-03	2025-12-03	2025-12-03
9	4	3.000000	2023-12-03	2026-12-03	2026-12-12

10	12	2.500000	2023-06-03	2028-12-03	2028-12-12
11	32	0.333333	2023-05-03	2025-12-03	2025-12-03
12	4	3.000000	2023-12-03	2026-12-03	2026-12-12
13	12	2.500000	2023-06-03	2028-12-03	2028-12-12
14	32	0.333333	2023-05-03	2025-12-03	2025-12-03
15	4	3.000000	2023-12-03	2026-12-03	2026-12-12
16	12	2.500000	2023-06-03	2028-12-03	2028-12-12
17	32	0.333333	2023-05-03	2025-12-03	2025-12-03
18	4	3.000000	2023-12-03	2026-12-03	2026-12-12
19	12	2.500000	2023-06-03	2028-12-03	2028-12-12
20	32	0.333333	2023-05-03	2025-12-03	2025-12-03
21	4	3.000000	2023-12-03	2026-12-03	2026-12-12
22	12	2.500000	2023-06-03	2028-12-03	2028-12-12
23	32	0.333333	2023-05-03	2025-12-03	2025-12-03
24	4	3.000000	2023-12-03	2026-12-03	2026-12-12
25	12	2.500000	2023-06-03	2028-12-03	2028-12-12
26	32	0.333333	2023-05-03	2025-12-03	2025-12-03
27	4	3.000000	2023-12-03	2026-12-03	2026-12-12
28	12	2.500000	2023-06-03	2028-12-03	2028-12-12
29	32	0.333333	2023-05-03	2025-12-03	2025-12-03
30	15	0.500000	2023-06-30	2030-06-30	2030-06-30

1.5.2 Cash flow matrices

All cash flows can be represented in a matrix with all possible cash flow dates as columns and all bonds as rows. The non-zero entries are the cash flow per unit of the bond at that date. The columns are the unique payment dates across the portfolio, from `unique_dates_profile`, computed separately for coupons and notional repayments:

[16]:
```
unique_list = bond_portfolio.unique_dates_profile(coupon_flows)
```

[17]:
```
unique_terminal_list = bond_portfolio.unique_dates_profile(notional_flows)
```

A dataframe of cash flows and notional amount payments is created:

[18]:
```
# Dataframe with bond  coupon cash flows
cash_flows = create_cashflow_dataframe(coupon_flows, unique_list)
# Dataframe with bond notional cash flows
notional_cash_flows = create_cashflow_dataframe(notional_flows,␣
 ↪unique_terminal_list)
```

Cash flow dataframe with coupon amounts and dates:

---

[19]:
```
display(cash_flows.head())
```

```
      2023-05-03  2023-06-03  2023-06-30  2023-07-03  2023-08-03  2023-09-03  \
1234    0.000000    0.000000         0.0    0.000000    0.000000    0.000000
```

```
2889    0.000000    2.500000         0.0    0.000000    0.000000    0.000000
31      0.333333    0.333333         0.0    0.333333    0.333333    0.333333
1       0.000000    0.000000         0.5    0.000000    0.000000    0.000000
2       0.000000    0.000000         0.0    0.000000    0.000000    0.000000

      2023-10-03  2023-11-03  2023-12-03  2023-12-30  …  2029-06-30  \
1234    0.000000    0.000000    3.000000         0.0  …         0.0
2889    0.000000    0.000000    2.500000         0.0  …         0.0
31      0.333333    0.333333    0.333333         0.0  …         0.0
1       0.000000    0.000000    0.000000         0.5  …         0.5
2       0.000000    0.000000    0.000000         0.0  …         0.0

      2029-12-30  2029-12-31  2030-06-30  2030-12-31  2031-12-31  2032-12-31  \
1234         0.0         0.0         0.0         0.0         0.0         0.0
2889         0.0         0.0         0.0         0.0         0.0         0.0
31           0.0         0.0         0.0         0.0         0.0         0.0
1            0.5         0.0         0.5         0.0         0.0         0.0
2            0.0         1.0         0.0         1.0         1.0         1.0

      2033-12-31  2034-12-31  2035-12-31
1234         0.0         0.0         0.0
2889         0.0         0.0         0.0
31           0.0         0.0         0.0
1            0.0         0.0         0.0
2            1.0         1.0         1.0

[5 rows x 66 columns]
```

Cash flow dataframe with notional amount payments and dates:

---

[20]:
```
display(notional_cash_flows.head())
```

```
      2025-12-03  2026-12-12  2028-12-12  2030-06-30  2035-12-31
1234         0.0       100.0         0.0         0.0         0.0
2889         0.0         0.0       100.0         0.0         0.0
31         100.0         0.0         0.0         0.0         0.0
1            0.0         0.0         0.0       100.0         0.0
2            0.0         0.0         0.0         0.0       100.0
```

1.6 5. Step 4 in numbers: pricing off the risk-free curve

This section prices the cash flows from section 4 off the risk-free curve (Step 4), first on its own and then with the input z-spread added.

1.6.1 The risk-free curve

import_SWEiopa() reads the necessary data about the current yield curve. The risk free term structure it produces is what every bond cash flow is discounted against, before the bond specific z-spread is added on top. Inside OSEM, the parameters related to the yield curve are saved in the Curves object.

[21]:
```
[maturities_country, curve_country, extra_param, Qb] = import_SWEiopa(settings.
 ↪EIOPA_param_file,
                                                                        ␣
 ↪settings.EIOPA_curves_file, settings.country)
# Curves object with information about term structure
curves = Curves(extra_param["UFR"] / 100, settings.precision, settings.tau,␣
 ↪settings.modelling_date,
                settings.country)
```

[22]:
```
ufr = extra_param["UFR"]/100 # ultimate forward rate
precision = float(settings.precision) # Numeric precision of the optimisation
# Targeted distance between the extrapolated curve and the ufr at the␣
 ↪convergence point
tau = float(settings.tau) # 1 basis point
```

[23]:
```
curves.set_observed_term_structure(maturity_vec=curve_country.index.
 ↪to_numpy(dtype=float), yield_vec=curve_country.values)
curves.calc_fwd_rates()
# The +1 matches main.py: the last projection period still has to be priced, so␣
 ↪the
# projected curve has to reach one year past the end of the modelling window.
curves.project_forward_rate(settings.n_proj_years+1)
curves.calibrate_projected(settings.n_proj_years+1, 0.05, 0.5, 1000)
```

1.6.2 Pricing state at the modelling date

`init_bond_portfolio_to_dataframe` sets up the three DataFrames the pricing works on, each with one column for the modelling date: the market price of each bond, its z-spread and the units held.

[24]:
```
[market_price_df, zspread_df, units_df] = bond_portfolio.
 ↪init_bond_portfolio_to_dataframe(settings.modelling_date)
```

At this point `zspread_df` holds the `Z_Spread` read from the portfolio file, the extra yield over the risk-free curve for the bond's credit and liquidity risk. The calibration in sections 6 and 7 overwrites it:

---

[25]:
```
display(zspread_df)
```

```
      2023-04-29
1234       0.010
```

```
2889    0.010
31      0.010
1       0.005
2       0.005
3       0.010
4       0.010
5       0.010
6       0.010
7       0.010
8       0.010
9       0.010
10      0.010
11      0.010
12      0.010
13      0.010
14      0.010
15      0.010
16      0.010
17      0.010
18      0.010
19      0.010
20      0.010
21      0.010
22      0.010
23      0.010
24      0.010
25      0.010
26      0.010
27      0.010
28      0.010
29      0.010
30      0.005
```

All pricing in this notebook happens at the modelling date, so the projection year is 0:

[26]:
```
proj_period = 0
```

1.6.3 Risk-free price: the gap the z-spread has to close

Pricing the cash flows at $z = 0$ gives MV_{rf} of Step 4, the value of the bond if it carried no credit or liquidity risk at all. The risk free price is what the cash flows are worth on the EIOPA curve alone, the market price is what the bond actually trades at, and the z-spread is the constant addition to the curve that closes the difference.

---

[27]:
```
risk_free_price = {}
for asset_id, bond in bond_portfolio.corporate_bonds.items():
    risk_free_price[asset_id] = bond.price_bond(cash_flows.loc[asset_id],
                                                notional_cash_flows.
 ↪loc[asset_id],
                                                settings.modelling_date,
                                                proj_period,
                                                curves,
                                                0.0)

price_comparison = pd.DataFrame({
    "Risk free price": pd.Series(risk_free_price),
    "Market price": pd.Series({aid: b.market_price
                               for aid, b in bond_portfolio.corporate_bonds.
 ↪items()}),
})
price_comparison["Gap to close"] = (price_comparison["Risk free price"]
                                    - price_comparison["Market price"])

display(price_comparison)
```

```
C:\Users\grego\anaconda3\Lib\site-packages\numpy\core\fromnumeric.py:86:
FutureWarning: The behavior of DataFrame.sum with axis=None is deprecated, in a
future version this will reduce over both axes and return a scalar. To retain
the old behavior, pass axis=0 (or do not pass axis)
  return reduction(axis=axis, out=out, **passkwargs)
C:\Users\grego\OneDrive\Documenti\GitHub\Open_Source_Economic_Model\src\osem\Bon
dClasses.py:241: FutureWarning: Calling float on a single element Series is
deprecated and will raise a TypeError in the future. Use float(ser.iloc[0])
instead
  disc_value: float = float(np.sum(nodisc_value))
```

	Risk free price	Market price	Gap to close
1234	100.921641	94.0	6.921641
2889	112.714855	92.0	20.714855
31	102.345972	96.0	6.345972
1	88.453864	90.0	-1.546136
2	80.528539	85.0	-4.471461
3	100.921641	94.0	6.921641
4	112.714855	92.0	20.714855
5	102.345972	96.0	6.345972
6	100.921641	94.0	6.921641
7	112.714855	92.0	20.714855
8	102.345972	96.0	6.345972
9	100.921641	94.0	6.921641
10	112.714855	92.0	20.714855
11	102.345972	96.0	6.345972
12	100.921641	94.0	6.921641

13	112.714855	92.0	20.714855
14	102.345972	96.0	6.345972
15	100.921641	94.0	6.921641
16	112.714855	92.0	20.714855
17	102.345972	96.0	6.345972
18	100.921641	94.0	6.921641
19	112.714855	92.0	20.714855
20	102.345972	96.0	6.345972
21	100.921641	94.0	6.921641
22	112.714855	92.0	20.714855
23	102.345972	96.0	6.345972
24	100.921641	94.0	6.921641
25	112.714855	92.0	20.714855
26	102.345972	96.0	6.345972
27	100.921641	94.0	6.921641
28	112.714855	92.0	20.714855
29	102.345972	96.0	6.345972
30	88.453864	90.0	-1.546136

1.6.4 Price at the input z-spread

The same cash flows discounted at the risk free curve **plus the input `Z_Spread`** held in `zspread_df`. These prices do not reproduce the observed `Market_Price` either; closing that gap is what the calibration in the next section does.

Note that `main.py` has no counterpart to this step. There the bond price at the modelling date is the `Market_Price` from the input file, and the first discounted cash flow price is produced inside the main loop.

[28]:
```
market_price_df = bond_portfolio.price_bond_portfolio(cash_flows,␣
↪notional_cash_flows, settings, proj_period, curves, zspread_df,␣
↪market_price_df,settings.modelling_date)
```

```
C:\Users\grego\anaconda3\Lib\site-packages\numpy\core\fromnumeric.py:86:
FutureWarning: The behavior of DataFrame.sum with axis=None is deprecated, in a
future version this will reduce over both axes and return a scalar. To retain
the old behavior, pass axis=0 (or do not pass axis)
  return reduction(axis=axis, out=out, **passkwargs)
C:\Users\grego\OneDrive\Documenti\GitHub\Open_Source_Economic_Model\src\osem\Bon
dClasses.py:241: FutureWarning: Calling float on a single element Series is
deprecated and will raise a TypeError in the future. Use float(ser.iloc[0])
instead
  disc_value: float = float(np.sum(nodisc_value))
```

---

[29]:
```
price_comparison.insert(1, "Price at input z-spread", market_price_df.iloc[:,␣
↪0])
```

```
display(price_comparison)
```

	Risk free price	Price at input z-spread	Market price	Gap to close
1234	100.921641	97.618258	94.0	6.921641
2889	112.714855	107.485399	92.0	20.714855
31	102.345972	99.940951	96.0	6.345972
1	88.453864	85.547108	90.0	-1.546136
2	80.528539	76.050676	85.0	-4.471461
3	100.921641	97.618258	94.0	6.921641
4	112.714855	107.485399	92.0	20.714855
5	102.345972	99.940951	96.0	6.345972
6	100.921641	97.618258	94.0	6.921641
7	112.714855	107.485399	92.0	20.714855
8	102.345972	99.940951	96.0	6.345972
9	100.921641	97.618258	94.0	6.921641
10	112.714855	107.485399	92.0	20.714855
11	102.345972	99.940951	96.0	6.345972
12	100.921641	97.618258	94.0	6.921641
13	112.714855	107.485399	92.0	20.714855
14	102.345972	99.940951	96.0	6.345972
15	100.921641	97.618258	94.0	6.921641
16	112.714855	107.485399	92.0	20.714855
17	102.345972	99.940951	96.0	6.345972
18	100.921641	97.618258	94.0	6.921641
19	112.714855	107.485399	92.0	20.714855
20	102.345972	99.940951	96.0	6.345972
21	100.921641	97.618258	94.0	6.921641
22	112.714855	107.485399	92.0	20.714855
23	102.345972	99.940951	96.0	6.345972
24	100.921641	97.618258	94.0	6.921641
25	112.714855	107.485399	92.0	20.714855
26	102.345972	99.940951	96.0	6.345972
27	100.921641	97.618258	94.0	6.921641
28	112.714855	107.485399	92.0	20.714855
29	102.345972	99.940951	96.0	6.345972
30	88.453864	85.547108	90.0	-1.546136

1.7 6. Step 5 in numbers: calibrating one bond

Step 5 for bond 1234: the bisection searches the bracket $[-0.2, 0.2]$ for the spread that, added on top of the risk free term structure, makes the discounted cash flows equal the market price.

[30]:
```
calibrated_spread = bond_portfolio.corporate_bonds[1234].
↪bisection_spread(x_start=-0.2
                                , x_end=0.2
```

```
                                , modelling_date=settings. modelling_date
                                , end_date=settings.end_date
                                , proj_period=proj_period
                                , curves=curves
                                , precision= 0.00000001
                                , max_iter=100000)
```

```
C:\Users\grego\anaconda3\Lib\site-packages\numpy\core\fromnumeric.py:86:
FutureWarning: The behavior of DataFrame.sum with axis=None is deprecated, in a
future version this will reduce over both axes and return a scalar. To retain
the old behavior, pass axis=0 (or do not pass axis)
  return reduction(axis=axis, out=out, **passkwargs)
C:\Users\grego\OneDrive\Documenti\GitHub\Open_Source_Economic_Model\src\osem\Bon
dClasses.py:241: FutureWarning: Calling float on a single element Series is
deprecated and will raise a TypeError in the future. Use float(ser.iloc[0])
instead
  disc_value: float = float(np.sum(nodisc_value))
```

The market value calculated using the discounted cash flow method using the calibrated zspread is:

[31]:
```
calibrated_bond = bond_portfolio.corporate_bonds[1234].price_bond(cash_flows.
↪loc[1234],notional_cash_flows.loc[1234],settings.modelling_date,␣
↪proj_period,curves,calibrated_spread)
```

---

[32]:
```
print(calibrated_bond)
```

```
94.00000089794409
```

1.8 7. Step 6 in numbers: calibrating the whole portfolio

`calibrate_bond_portfolio` repeats the bisection for every bond (Step 6) and writes the results into `zspread_df`:

[33]:
```
zspread_df=bond_portfolio.calibrate_bond_portfolio(zspread_df, settings,␣
↪proj_period, curves)
```

---

[34]:
```
display(zspread_df)
```

```
      2023-04-29
1234    0.021480
2889    0.043776
31      0.027160
1      -0.002582
2      -0.004688
```

```
3       0.021480
4       0.043776
5       0.027160
6       0.021480
7       0.043776
8       0.027160
9       0.021480
10      0.043776
11      0.027160
12      0.021480
13      0.043776
14      0.027160
15      0.021480
16      0.043776
17      0.027160
18      0.021480
19      0.043776
20      0.027160
21      0.021480
22      0.043776
23      0.027160
24      0.021480
25      0.043776
26      0.027160
27      0.021480
28      0.043776
29      0.027160
30     -0.002582
```

Repricing the portfolio with the calibrated spreads reproduces the observed market prices. The residual is the price cost of stopping the bisection once the spread bracket is narrower than `precision`, as described in Step 5:

---

[35]:
```
calibrated_price_df = bond_portfolio.price_bond_portfolio(cash_flows,
                                                          notional_cash_flows,
                                                          settings,
                                                          proj_period,
                                                          curves,
                                                          zspread_df,
                                                          market_price_df.copy(),
                                                          settings.
 ↪modelling_date)

calibration_check = pd.DataFrame({
    "Calibrated z-spread": zspread_df.iloc[:, 0],
```

```
    "Repriced value": calibrated_price_df.iloc[:, 0],
    "Market price": pd.Series({aid: bond.market_price
                               for aid, bond in bond_portfolio.corporate_bonds.
 ↪items()}),
})
calibration_check["Residual"] = (calibration_check["Repriced value"]
                                 - calibration_check["Market price"])

display(calibration_check)
```

```
C:\Users\grego\anaconda3\Lib\site-packages\numpy\core\fromnumeric.py:86:
FutureWarning: The behavior of DataFrame.sum with axis=None is deprecated, in a
future version this will reduce over both axes and return a scalar. To retain
the old behavior, pass axis=0 (or do not pass axis)
  return reduction(axis=axis, out=out, **passkwargs)
C:\Users\grego\OneDrive\Documenti\GitHub\Open_Source_Economic_Model\src\osem\Bon
dClasses.py:241: FutureWarning: Calling float on a single element Series is
deprecated and will raise a TypeError in the future. Use float(ser.iloc[0])
instead
  disc_value: float = float(np.sum(nodisc_value))
```

	Calibrated z-spread	Repriced value	Market price	Residual
1234	0.021480	94.000001	94.0	8.979441e-07
2889	0.043776	92.000001	92.0	1.122554e-06
31	0.027160	96.000000	96.0	-3.087680e-07
1	-0.002582	90.000002	90.0	1.737471e-06
2	-0.004688	84.999995	85.0	-4.696790e-06
3	0.021480	94.000001	94.0	8.979441e-07
4	0.043776	92.000001	92.0	1.122554e-06
5	0.027160	96.000000	96.0	-3.087680e-07
6	0.021480	94.000001	94.0	8.979441e-07
7	0.043776	92.000001	92.0	1.122554e-06
8	0.027160	96.000000	96.0	-3.087680e-07
9	0.021480	94.000001	94.0	8.979441e-07
10	0.043776	92.000001	92.0	1.122554e-06
11	0.027160	96.000000	96.0	-3.087680e-07
12	0.021480	94.000001	94.0	8.979441e-07
13	0.043776	92.000001	92.0	1.122554e-06
14	0.027160	96.000000	96.0	-3.087680e-07
15	0.021480	94.000001	94.0	8.979441e-07
16	0.043776	92.000001	92.0	1.122554e-06
17	0.027160	96.000000	96.0	-3.087680e-07
18	0.021480	94.000001	94.0	8.979441e-07
19	0.043776	92.000001	92.0	1.122554e-06
20	0.027160	96.000000	96.0	-3.087680e-07
21	0.021480	94.000001	94.0	8.979441e-07
22	0.043776	92.000001	92.0	1.122554e-06
23	0.027160	96.000000	96.0	-3.087680e-07

24	0.021480	94.000001	94.0	8.979441e-07
25	0.043776	92.000001	92.0	1.122554e-06
26	0.027160	96.000000	96.0	-3.087680e-07
27	0.021480	94.000001	94.0	8.979441e-07
28	0.043776	92.000001	92.0	1.122554e-06
29	0.027160	96.000000	96.0	-3.087680e-07
30	-0.002582	90.000002	90.0	1.737471e-06

1.9 8. Appendix: input data

The input files this notebook reads, all in the `Input` folder. Their locations come from the `[INPUT]` section of `ALM.ini`, except the EIOPA files, whose paths are set in `Parameters.csv`. Dates are written as `dd/mm/yyyy`.

1.9.1 Parameters.csv

The type of run and its parameters, one `Parameter`, `Value` pair per row. Loaded into `Settings` by `get_settings`.

Parameter	Example	Meaning
`EIOPA_param_file`	`Input/Param_no_VA.csv`	Relative path of the EIOPA parameter file
`EIOPA_curves_file`	`Input/Curves_no_VA.csv`	Relative path of the EIOPA spot curve file
`country`	`Slovenia`	Country whose curve is used; must match a column name in both EIOPA files
`run_type`	`Risk Neutral`	Type of run. Read into `Settings` but not used yet
`n_proj_years`	`50`	Length of the run in years. The end of the modelling window E is the modelling date plus this many years
`Precision`	`1E-10`	Precision of the Smith-Wilson calibration in `Curves`. The bond z-spread calibration does not use it: `calibrate_bond_portfolio` hardcodes `1e-8`
`Tau`	`0.0001`	Accepted gap between the extrapolated forward rate and the UFR at the convergence point (1 bp)

Parameter	Example	Meaning
`compounding`	`-1`	Compounding convention. Read into `Settings` but not used yet; discounting is always annual, $(1+y)^{-t}$
`Modelling_Date`	`29/04/2023`	Start date of the run, MD
`liability_mode`	`cashflow`	`cashflow` or `unit_linked`; selects the liability model of the full run. Not used in this notebook
`random_seed`	`42`	Seed for the random draws in `unit_linked` mode. Not used in this notebook

1.9.2 EIOPA RFR files

Two files derived from EIOPA's monthly risk-free rate (RFR) publication. `import_SWEiopa` reads the columns of the chosen `country` from each. The `no_VA` files hold the curves without the volatility adjustment.

- **`Curves_no_VA.csv`**: spot rates, one row per maturity (1 to 150 years) and one column per country, as decimals (`0.03472` is 3.472%).
- **`Param_no_VA.csv`**: the parameters EIOPA used to build the curves, in two columns per country (`<Country>_Maturities`, `<Country>_Values`). The first six rows are scalar parameters: `Coupon_freq`, `LLP` (last liquid point), `Convergence` (convergence period), `UFR` (ultimate forward rate, in percent, e.g. `3.45`), `alpha` (speed of convergence to the UFR) and `CRA` (credit risk adjustment, in bp). The rows after them hold the Smith-Wilson calibration vector by maturity.

Together they give the risk free term structure at the modelling date, from which `Curves` projects its evolution over the run.

1.9.3 Bond_Portfolio.csv

The fixed income portfolio, one row per bond position. Loaded by `get_corporate_bonds`, one `CorpBond` per row. Section 3 shows which of these fields enter the price.

Column	Example	Meaning
`Asset_ID`	`1234`	Unique integer id of the position
`Asset_Type`	`Corporate_Bond`	Asset type label; not read by `get_corporate_bonds`
`NACE`	`A1.4.5`	NACE code of the issuer (*nomenclature statistique des activités économiques dans la Communauté européenne*)
`Issue_Date`	`3/12/2021`	Issue date I; anchors the coupon grid

Column	Example	Meaning
`Maturity_Date`	`12/12/2026`	Maturity date M; the notional is repaid on it
`Notional_Amount`	`100`	Notional amount N
`Coupon_Rate`	`0.03`	Coupon c paid on **each** coupon date, as a fraction of the notional. Not annualised: see the note on f under Notation
`Spread_Country`	`0`	Country spread; not used in bond pricing
`Spread_Sector`	`0`	Sector spread; not used in bond pricing
`Z_Spread`	`0.01`	Starting z-spread; replaced by the calibrated value
`Spread_Stress`	`0`	Spread added in a stress scenario; not used in bond pricing
`Frequency`	`1`	Coupon payments per year f: 1, 2, 3, 4 or 12
`Recovery_Rate`	`0.4`	Fraction of the notional recovered on default; not used yet
`Default_Probability`	`0.03`	Annual probability of default; not used yet
`Units`	`100`	Number of bonds held
`Market_Price`	`94`	Price of one bond at the modelling date, on the same scale as `Notional_Amount`; the calibration target MV

1.9.4 Sector_Spread.csv

Sector spreads by NACE code. In this proof of concept the bond spread comes from `Bond_Portfolio.csv`, so this file is loaded in section 3 for reference only.

Column	Example	Meaning
`NACE`	`A1`	NACE code, at section, division or group level
`NACE code text`	`Crop and animal production, hunting and related service activities`	Description of the NACE code
`sSpread`	`0.01`	Sector specific spread over the risk free rate